



VIT-AP UNIVERSITY

CINEMATCH - MOVIE RECOMMENDATION AND ANALYTICS SYSTEM

A NoSQL Database Case Study Using MongoDB

Registration Number	Name
24MIC7198	V. RUTHVIK
24MIC7181	K. TEJASH NARAYANA
24MIC7203	B. HARSHITH

Under The Guidance Of : Dr. VASAVI SANIKOMMU

School of Computer Science and Engineering (SCOPE)

VIT-AP University, Amaravati

Problem Statement

Movie streaming platforms collect a large number of ratings from their users. This rating data is useful to find out which movies are popular, which genres are liked by the users, how active the users are and what movies a user may like to watch next. The MovieLens dataset contains this kind of data in four separate CSV files (movies, ratings, tags and links). The genre information of a movie is stored as a single string separated by the pipe symbol, and the ratings of a movie are stored in a different file, so answering simple questions like "what is the average rating of each genre" needs the files to be combined and processed.

In this project we use MongoDB, a document based NoSQL database, to store the MovieLens data and analyse it. The data is imported as collections, cleaned inside MongoDB, and then analysed using find queries and aggregation pipelines. The project also builds a simple recommendation feature using the same queries and shows the results in a web application.

Aim

The aim of this project is to store the MovieLens movie rating dataset in MongoDB and to analyse it using MongoDB queries and aggregation pipelines. The project focuses on finding information about movies, genres, ratings, users and tags, and on recommending movies to users using the rating data.

Objectives

1. To import the MovieLens CSV files into MongoDB as separate collections.
2. To clean the imported data inside MongoDB (extract the release year, split the genres string into an array and convert timestamps to dates).
3. To perform basic CRUD operations and queries using filters, projection, sorting, regular expressions and text search.
4. To analyse the ratings: rating distribution, most rated movies, highest rated movies and most controversial movies.
5. To analyse genres by joining the ratings and movies collections using \$lookup and \$unwind.
6. To analyse users: most active users, harsh and generous users, and rating activity over the years.
7. To analyse the tags given by users.
8. To recommend movies using genre similarity, "users who liked this also liked" and weighted rating.
9. To create indexes and check their effect using explain().
10. To display the results in a web application connected to MongoDB.

Dataset Description

The project uses the MovieLens "ml-latest-small" dataset published by GroupLens Research, University of Minnesota. The same dataset is available on Kaggle as "Popular Movies Datasets - 9742 Movies". It contains 100,836 ratings and 3,683 tags given by 610 users to 9,742 movies between 1996 and 2018. Every user has rated at least 20 movies. The dataset has four CSV files:

File	Rows	Columns	Description
movies.csv	9,742	movieId, title, genres	Movie title with the release year in brackets and genres separated by
ratings.csv	100,836	userId, movieId, rating, timestamp	Rating given by a user to a movie (0.5 to 5.0 stars) with Unix timestamp
tags.csv	3,683	userId, movieId, tag, timestamp	Free text tag given by a user to a movie
links.csv	9,742	movieId, imdbId, tmdbId	IMDb and TMDb ids of each movie

Sample rows from the files:

```
movies.csv
movieId,title,genres
1,Toy Story (1995),Adventure|Animation|Children|Comedy|Fantasy
2,Jumanji (1995),Adventure|Children|Fantasy
```

```
ratings.csv
userId,movieId,rating,timestamp
1,1,4.0,964982703
1,3,4.0,964981247
```

```
tags.csv
userId,movieId,tag,timestamp
2,60756,funny,1445714994
```

```
links.csv
movieId,imdbId,tmdbId
1,0114709,862
```

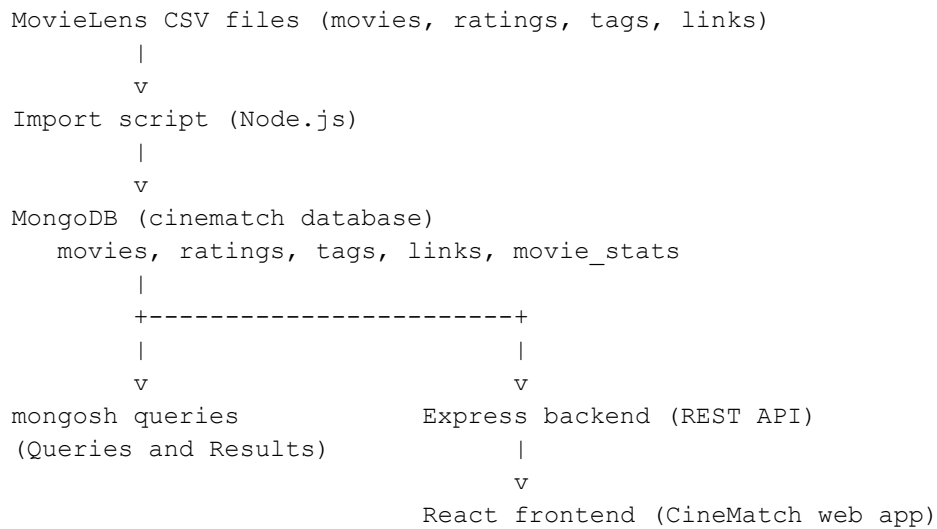
Why MongoDB

- A movie has many genres. In MongoDB this is stored as an array inside the movie document, so there is no need for a separate genre table and join.
- The data could be cleaned inside the database itself using updateMany with pipeline stages, without writing a separate program.
- The aggregation pipeline (\$match, \$group, \$sort, \$lookup, \$unwind, \$facet etc.) can answer complex questions in a single query.
- Ratings refer to movies through movieId (references), while genres are embedded in the movie document. Both NoSQL data modelling styles are used.
- Indexes on movieId, userId, the genre array and the title keep the queries fast, which was checked with explain().

Technology Stack

Part	Technology
Database	MongoDB Community Server 8.3
Database Shell	mongosh
Data Import	Node.js script using csv-parse and the MongoDB Node.js driver
Backend	Node.js, Express
Frontend	React (Vite), Recharts
Code Editor	Visual Studio Code

System Architecture



The CSV files are imported into four collections. Cleaning queries add the year, genre array and date fields. The same queries are run in mongosh for this report and are also used by the Express backend, which sends the results to the React frontend.

MongoDB Database Design

Database name: cinematch

Collection	Documents	Description
movies	9,742	Movie details. Fields: movieId, title, genres, year, genreList
ratings	100,836	Ratings given by users. Fields: userId, movieId, rating, timestamp, ratedAt
tags	3,683	Tags given by users. Fields: userId, movieId, tag, timestamp, taggedAt
links	9,742	IMDb and TMDb ids. Fields: movieId, imdbId, tmdbId
movie_stats	9,724	Rating count, average and standard deviation of every movie (created with \$merge, used by the web application)

year, genreList, ratedAt and taggedAt are new fields added by the cleaning queries.

Relationships

- ratings.movieId, tags.movieId and links.movieId refer to movies.movieId. They are joined using \$lookup.
- movie_stats._id is the movieId of the movie.
- There is no users collection because the dataset does not have user details. User information is calculated from the ratings collection.

Ratings are kept in a separate collection because a popular movie has hundreds of ratings and ratings are also queried by user and by date. Genres are embedded in the movie document as an array because they are small and always needed along with the movie.

Sample documents after cleaning

```
movies
{
  _id: ObjectId('...'),
```

```

    movieId: 1,
    title: 'Toy Story (1995)',
    genres: 'Adventure|Animation|Children|Comedy|Fantasy',
    year: 1995,
    genreList: [ 'Adventure', 'Animation', 'Children', 'Comedy', 'Fantasy' ]
  }

ratings
{ userId: 1, movieId: 1, rating: 4, timestamp: 964982703,
  ratedAt: ISODate('2000-07-30T18:45:03.000Z') }

tags
{ userId: 2, movieId: 60756, tag: 'funny', timestamp: 1445714994,
  taggedAt: ISODate('2015-10-24T19:29:54.000Z') }

movie_stats
{ _id: 356, ratingCount: 329, avgRating: 4.164, stdDev: 0.829 }

```

Indexes

Collection	Index	Purpose
ratings	{ movieId: 1 }	Joining ratings with movies
ratings	{ userId: 1 }	Queries by user
tags	{ movieId: 1 }	Tags of a movie
movies	{ movieId: 1 } (unique)	Primary lookup key
movies	{ genreList: 1 } (multikey)	Filtering by genre
movies	{ year: 1 }	Filtering by year
movies	{ title: 'text' }	Text search on titles

Implementation

In this project we used MongoDB as the NoSQL database to store and analyse the MovieLens data. The four CSV files were imported into the cinematch database as separate collections using a Node.js import script, which works the same way as mongoimport with the --headerline option. After importing, the collections and sample documents were checked in mongosh.

The imported data was then cleaned inside MongoDB. The release year was taken out of the title using \$regexFind, the genres string was split into an array using \$split, titles ending with ", The" were rewritten in the normal order, and the Unix timestamps were converted to dates using \$toDate. Indexes were created on the fields used for joining and filtering.

After this, different MongoDB operations were used to analyse the data. Basic find queries with filters, projection, sorting, regular expressions and text search were used to look up movies. Aggregation pipelines with \$group, \$lookup, \$unwind, \$sort, \$facet, \$bucket and date operators were used to analyse ratings, genres, users and tags. Recommendation queries were written using \$setIntersection (genre similarity), a \$lookup on the fans of a movie (users who liked this also liked) and a weighted rating formula. Finally, explain() was used to show that the queries use the indexes.

All the queries were executed in mongosh and the output of each query is shown as a screenshot in the next section. The same queries are used by the backend of the CineMatch web application, which is shown at the end.

Queries and Results

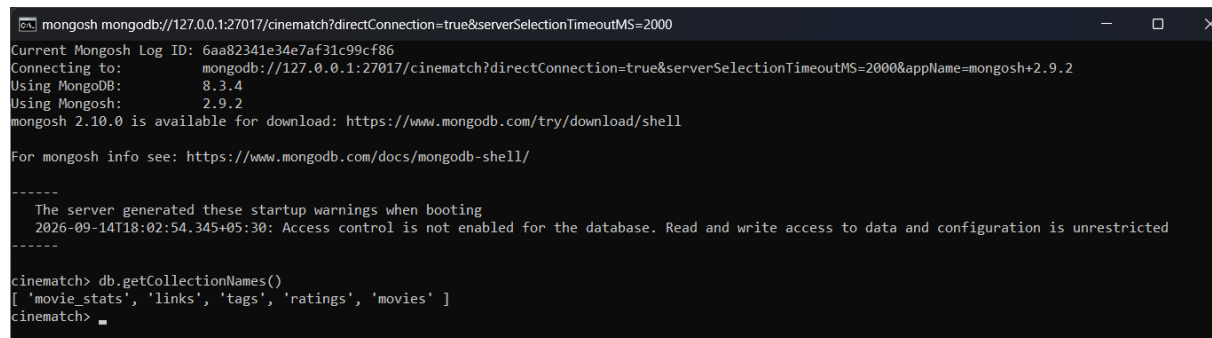
Database Overview

1. List all collections in the cinematch database

Query:

```
db.getCollectionNames()
```

Output:



```
mongosh mongodb://127.0.0.1:27017/cinematch?directConnection=true&serverSelectionTimeoutMS=2000
Current Mongosh Log ID: 6aa82341e34e7af31c99cf86
Connecting to:      mongodb://127.0.0.1:27017/cinematch?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.9.2
Using MongoDB:      8.3.4
Using Mongosh:      2.9.2
mongosh 2.10.0 is available for download: https://www.mongodb.com/try/download/shell
For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
  The server generated these startup warnings when booting
  2026-09-14T18:02:54.345+05:30: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----

cinematch> db.getCollectionNames()
[ 'movie_stats', 'links', 'tags', 'ratings', 'movies' ]
cinematch>
```

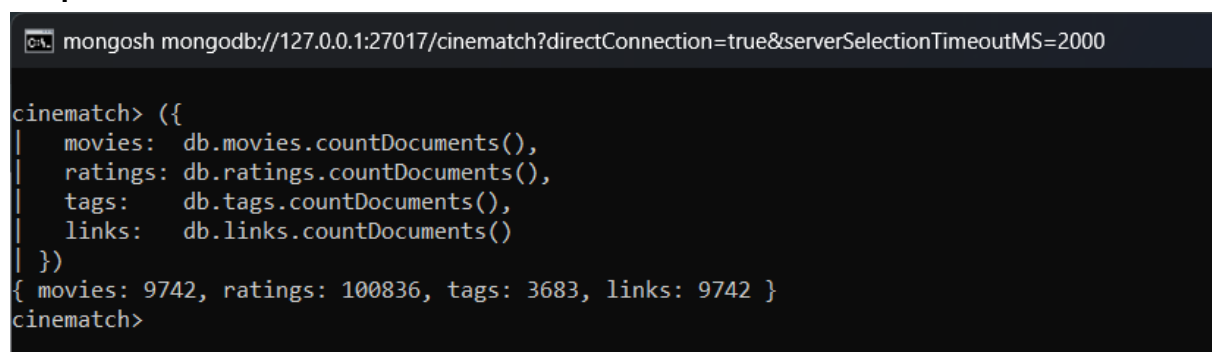
Result: The four collections created from the CSV files are listed (movie_stats is added later by Query 53).

2. Count documents in every collection

Query:

```
{
  movies: db.movies.countDocuments(),
  ratings: db.ratings.countDocuments(),
  tags: db.tags.countDocuments(),
  links: db.links.countDocuments()
}
```

Output:



```
mongosh mongodb://127.0.0.1:27017/cinematch?directConnection=true&serverSelectionTimeoutMS=2000

cinematch> ({
|   movies: db.movies.countDocuments(),
|   ratings: db.ratings.countDocuments(),
|   tags: db.tags.countDocuments(),
|   links: db.links.countDocuments()
| })
{ movies: 9742, ratings: 100836, tags: 3683, links: 9742 }
cinematch>
```

Result: The document counts match the dataset: 9,742 movies, 100,836 ratings, 3,683 tags and 9,742 links.

3. Display a sample movie document

Query:

```
db.movies.findOne({ movieId: 1 })
```

Output:

```
mongosh mongodb://127.0.0.1:27017/cinematch?directConnection=true&serverSelectionTimeoutMS=2000

cinematch> db.movies.findOne({ movieId: 1 })
{
  _id: ObjectId('6aa8233e44a9a8c130058ac1'),
  movieId: 1,
  title: 'Toy Story (1995)',
  genres: 'Adventure|Animation|Children|Comedy|Fantasy'
}
cinematch>
```

Result: The genres of the movie are stored as a single string separated by the pipe symbol.

Data Cleaning & Schema Enrichment

4. Extract the release year from the movie title

Query:

```
db.movies.updateMany({}, [
  {
    $set: {
      year: {
        $let: {
          vars: { m: { $regexFind: { input: "$title", regex:
/\\((\\d{4})\\)\\s*$ / } } },
          in: {
            $cond: [
              { $eq: ["$m", null] },
              null,
              { $toInt: { $arrayElemAt: ["$m.captures", 0] } }
            ]
          }
        }
      }
    }
  }
])
```

Output:

```
mongosh mongodb://127.0.0.1:27017/cinematch?directConnection=true&serverSelectionTimeoutMS=2000

cinematch> db.movies.updateMany({}, [
|   {
|     $set: {
|       year: {
|         $let: {
|           vars: { m: { $regexFind: { input: "$title", regex: /\((\d{4})\)\s*/ } } },
|           in: {
|             $cond: [
|               { $eq: ["$$m", null] },
|               null,
|               { $toInt: { $arrayElemAt: ["$$m.captures", 0] } }
|             ]
|           }
|         }
|       }
|     }
|   }
| ])
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 9742,
  modifiedCount: 9742,
  upsertedCount: 0
}
cinematch> 
```

Result: All 9,742 movie documents were updated with a numeric year field.

5. Split the pipe-delimited genres string into an array

Query:

```
db.movies.updateMany({}, [
  { $set: { genreList: { $split: ["$genres", "|"] } } }
])
```

Output:

```
mongosh mongodb://127.0.0.1:27017/cinematch?directConnection=true&serverSelectionTimeoutMS=2000

cinematch> db.movies.updateMany({}, [
|   { $set: { genreList: { $split: ["$genres", "|"] } } }
| ])
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 9742,
  modifiedCount: 9742,
  upsertedCount: 0
}
cinematch> 
```

Result: genreList is now an array, so the genres can be filtered and unwound individually.

6. Convert Unix timestamps into ISODate fields

Query:

```
db.ratings.updateMany({}, [
  { $set: { ratedAt: { $toDate: { $multiply: ["$timestamp", 1000] } } } }
]);
db.tags.updateMany({}, [
  { $set: { taggedAt: { $toDate: { $multiply: ["$timestamp", 1000] } } } }
])
```

Output:

```
mongosh mongodb://127.0.0.1:27017/cinematch?directConnection=true&serverSelectionTimeoutMS=2000

cinematch> db.ratings.updateMany({}, [
|   { $set: { ratedAt: { $toDate: { $multiply: ["$timestamp", 1000] } } } }
| ]);
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 100836,
  modifiedCount: 100836,
  upsertedCount: 0
}
cinematch> db.tags.updateMany({}, [
|   { $set: { taggedAt: { $toDate: { $multiply: ["$timestamp", 1000] } } } }
| ]);
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 3683,
  modifiedCount: 3683,
  upsertedCount: 0
}
cinematch> _
```

Result: All ratings and tags now have an ISODate field which can be used with \$year and \$dayOfWeek.

7. Normalise titles that end with a trailing article

Query:

```
db.movies.updateMany(
  { title: /, (The|A|An) \(/ },
  [
    {
      $set: {
        title: {
          $let: {
            vars: { m: { $regexFind: { input: "$title", regex: /^(.+),
(The|A|An) (\(.+)\$/ } } },
            in: {
              $concat: [
                { $arrayElemAt: ["$$m.captures", 1] }, " ",
                { $arrayElemAt: ["$$m.captures", 0] }, " ",
                { $arrayElemAt: ["$$m.captures", 2] }
              ]
            }
          }
        }
      }
    }
  ]
)
```

```

    }
  }
}
]
)

```

Output:

```

C:\> mongosh mongodb://127.0.0.1:27017/cinematch?directConnection=true&serverSelectionTimeoutMS=2000

cinematch> db.movies.updateMany(
  { title: /, (The|A|An) \(/ },
  [
    {
      $set: {
        title: {
          $let: {
            vars: { m: { $regexFind: { input: "$title", regex: /^(.+), (The|A|An) (\(.+)\$/ } } },
            in: {
              $concat: [
                { $arrayElemAt: ["$$m.captures", 1] }, " ",
                { $arrayElemAt: ["$$m.captures", 0] }, " ",
                { $arrayElemAt: ["$$m.captures", 2] }
              ]
            }
          }
        }
      }
    }
  ]
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1822,
  modifiedCount: 1822,
  upsertedCount: 0
}
cinematch>

```

Result: 1,822 titles were rewritten, for example 'Shawshank Redemption, The (1994)' became 'The Shawshank Redemption (1994)'.

8. Create indexes on frequently queried fields

Query:

```

db.ratings.createIndex({ movieId: 1 });
db.ratings.createIndex({ userId: 1 });
db.tags.createIndex({ movieId: 1 });
db.movies.createIndex({ movieId: 1 }, { unique: true });
db.movies.createIndex({ genreList: 1 });
db.movies.createIndex({ year: 1 });
db.movies.createIndex({ title: "text" });
db.movies.getIndexes().map(i => i.name)

```

Output:

```
mongosh mongodb://127.0.0.1:27017/cinematch?directConnection=true&serverSelectionTimeoutMS=2000

cinematch> db.ratings.createIndex({ movieId: 1 });
movieId_1
cinematch> db.ratings.createIndex({ userId: 1 });
userId_1
cinematch> db.tags.createIndex({ movieId: 1 });
movieId_1
cinematch> db.movies.createIndex({ movieId: 1 }, { unique: true });
movieId_1
cinematch> db.movies.createIndex({ genreList: 1 });
genreList_1
cinematch> db.movies.createIndex({ year: 1 });
year_1
cinematch> db.movies.createIndex({ title: "text" });
title_text
cinematch> db.movies.getIndexes().map(i => i.name)
[ '_id_', 'movieId_1', 'genreList_1', 'year_1', 'title_text' ]
cinematch>
```

Result: The indexes on the movies collection are listed, including the multikey index on genreList and the text index on title.

Movie Exploration

9. Search movie titles with a regular expression

Query:

```
db.movies.find({ title: { $regex: "star wars", $options: "i" } }, { _id: 0,
title: 1, year: 1 }).sort({ year: 1 })
```

Output:

```
mongosh mongodb://127.0.0.1:27017/cinematch?directConnection=true&serverSelectionTimeoutMS=2000

cinematch> db.movies.find({ title: { $regex: "star wars", $options: "i" } }, { _id: 0, title: 1, year: 1 }).sort({ year: 1 })
[
  { title: 'Star Wars: Episode IV - A New Hope (1977)', year: 1977 },
  { title: 'The Star Wars Holiday Special (1978)', year: 1978 },
  {
    title: 'Star Wars: Episode V - The Empire Strikes Back (1980)',
    year: 1980
  },
  {
    title: 'Star Wars: Episode VI - Return of the Jedi (1983)',
    year: 1983
  },
  {
    title: 'Star Wars: Episode I - The Phantom Menace (1999)',
    year: 1999
  },
  {
    title: 'Star Wars: Episode II - Attack of the Clones (2002)',
    year: 2002
  },
  {
    title: 'Empire of Dreams: The Story of the 'Star Wars' Trilogy (2004)',
    year: 2004
  },
  {
    title: 'Star Wars: Episode III - Revenge of the Sith (2005)',
    year: 2005
  },
  { title: 'Star Wars: The Clone Wars (2008)', year: 2008 },
  {
    title: 'Star Wars: Episode VII - The Force Awakens (2015)',
    year: 2015
  },
  { title: 'Rogue One: A Star Wars Story (2016)', year: 2016 },
  { title: 'Star Wars: The Last Jedi (2017)', year: 2017 },
  { title: 'Solo: A Star Wars Story (2018)', year: 2018 }
]
cinematch>
```

Result: 13 movies contain 'star wars' in the title. They are shown in order of year.

10. Number of movies per genre

Query:

```
db.movies.aggregate([
  { $unwind: "$genreList" },
  { $group: { _id: "$genreList", movieCount: { $sum: 1 } } },
  { $sort: { movieCount: -1 } }
])
```

Output:

```
mongosh mongodb://127.0.0.1:27017/cinematch?directConnection=true&serverSelectionTimeoutMS=2000

cinematch> db.movies.aggregate([
|   { $unwind: "$genreList" },
|   { $group: { _id: "$genreList", movieCount: { $sum: 1 } } },
|   { $sort: { movieCount: -1 } }
| ])
[
  { _id: 'Drama', movieCount: 4361 },
  { _id: 'Comedy', movieCount: 3756 },
  { _id: 'Thriller', movieCount: 1894 },
  { _id: 'Action', movieCount: 1828 },
  { _id: 'Romance', movieCount: 1596 },
  { _id: 'Adventure', movieCount: 1263 },
  { _id: 'Crime', movieCount: 1199 },
  { _id: 'Sci-Fi', movieCount: 980 },
  { _id: 'Horror', movieCount: 978 },
  { _id: 'Fantasy', movieCount: 779 },
  { _id: 'Children', movieCount: 664 },
  { _id: 'Animation', movieCount: 611 },
  { _id: 'Mystery', movieCount: 573 },
  { _id: 'Documentary', movieCount: 440 },
  { _id: 'War', movieCount: 382 },
  { _id: 'Musical', movieCount: 334 },
  { _id: 'Western', movieCount: 167 },
  { _id: 'IMAX', movieCount: 158 },
  { _id: 'Film-Noir', movieCount: 87 },
  { _id: '(no genres listed)', movieCount: 34 }
]
cinematch>
```

Result: Drama (4,361) and Comedy (3,756) have the most movies. Film-Noir (87) has the least.

11. Number of movies released per decade

Query:

```
db.movies.aggregate([
  { $match: { year: { $ne: null } } },
  {
    $group: {
      _id: { $multiply: [{ $floor: { $divide: ["$year", 10] } }, 10] },
      movies: { $sum: 1 }
    }
  },
  { $sort: { _id: 1 } }
])
```

Output:

```
mongosh mongodb://127.0.0.1:27017/cinematch?directConnection=true&serverSelectionTimeoutMS=2000

cinematch> db.movies.aggregate([
  { $match: { year: { $ne: null } } },
  {
    $group: {
      _id: { $multiply: [{ $floor: { $divide: ["$year", 10] } }, 10] },
      movies: { $sum: 1 }
    }
  },
  { $sort: { _id: 1 } }
])
[
  { _id: 1900, movies: 3 },
  { _id: 1910, movies: 7 },
  { _id: 1920, movies: 37 },
  { _id: 1930, movies: 136 },
  { _id: 1940, movies: 197 },
  { _id: 1950, movies: 279 },
  { _id: 1960, movies: 401 },
  { _id: 1970, movies: 500 },
  { _id: 1980, movies: 1177 },
  { _id: 1990, movies: 2212 },
  { _id: 2000, movies: 2849 },
  { _id: 2010, movies: 1931 }
]
cinematch> _
```

Result: The 1990s and 2000s have more than 2,000 movies each. Very few movies are before 1950.

12. Full-text search on movie titles

Query:

```
db.movies.find({ $text: { $search: "lord rings" } }, { _id: 0, title: 1,
score: { $meta: "textScore" } }).sort({ score: { $meta: "textScore" }
}).limit(5)
```

Output:

```
mongosh mongodb://127.0.0.1:27017/cinematch?directConnection=true&serverSelectionTimeoutMS=2000

cinematch> db.movies.find({ $text: { $search: "lord rings" } }, { _id: 0, title: 1, score: { $meta: "textScore" }
}).sort({ score: { $meta: "textScore" } }).limit(5)
MongoServerError[IndexNotFound]: text index required for $text query
cinematch>
```

Result: The text search returns the Lord of the Rings movies first, with their text score.

Rating Analysis

13. Rating distribution (how often each star value is used)

Query:

```
db.ratings.aggregate([
  { $group: { _id: "$rating", count: { $sum: 1 } } },
  { $sort: { _id: 1 } }
])
```

Output:

```
mongosh mongodb://127.0.0.1:27017/cinematch?directConnection=true&serverSelectionTimeoutMS=2000

cinematch> db.ratings.aggregate([
|   { $group: { _id: "$rating", count: { $sum: 1 } } },
|   { $sort: { _id: 1 } }
| ])
[
|   { _id: 0.5, count: 1370 },
|   { _id: 1, count: 2811 },
|   { _id: 1.5, count: 1791 },
|   { _id: 2, count: 7551 },
|   { _id: 2.5, count: 5550 },
|   { _id: 3, count: 20047 },
|   { _id: 3.5, count: 13136 },
|   { _id: 4, count: 26818 },
|   { _id: 4.5, count: 8551 },
|   { _id: 5, count: 13211 }
| ]
cinematch> _
```

Result: 4.0 is the most common rating (26,818 times), followed by 3.0 and 5.0. Whole star ratings are used more than half star ratings.

14. Overall rating statistics

Query:

```
db.ratings.aggregate([
|   {
|     $group: {
|       _id: null,
|       totalRatings: { $sum: 1 },
|       avgRating: { $avg: "$rating" },
|       minRating: { $min: "$rating" },
|       maxRating: { $max: "$rating" },
|       stdDev: { $stdDevPop: "$rating" }
|     }
|   },
|   {
|     $project: {
|       _id: 0,
|       totalRatings: 1,
|       avgRating: { $round: ["$avgRating", 3] },
|       minRating: 1,
|       maxRating: 1,
|       stdDev: { $round: ["$stdDev", 3] }
|     }
|   }
| ])
```

Output:

```
mongosh mongodb://127.0.0.1:27017/cinematch?directConnection=true&serverSelectionTimeoutMS=2000

cinematch> db.ratings.aggregate([
  {
    $group: {
      _id: null,
      totalRatings: { $sum: 1 },
      avgRating: { $avg: "$rating" },
      minRating: { $min: "$rating" },
      maxRating: { $max: "$rating" },
      stdDev: { $stdDevPop: "$rating" }
    }
  },
  {
    $project: {
      _id: 0,
      totalRatings: 1,
      avgRating: { $round: ["$avgRating", 3] },
      minRating: 1,
      maxRating: 1,
      stdDev: { $round: ["$stdDev", 3] }
    }
  }
])
[
  {
    totalRatings: 100836,
    minRating: 0.5,
    maxRating: 5,
    avgRating: 3.502,
    stdDev: 1.043
  }
]
cinematch>
```

Result: The overall average rating is 3.50 with a standard deviation of 1.04.

15. Top 5 most-rated (most popular) movies

Query:

```
db.ratings.aggregate([
  {
    $group: {
      _id: "$movieId",
      ratingCount: { $sum: 1 },
      avgRating: { $avg: "$rating" }
    }
  },
  { $sort: { ratingCount: -1 } },
  { $limit: 5 },
  {
    $lookup: {
      from: "movies",
      localField: "_id",
      foreignField: "movieId",
      as: "movie"
    }
  }
])
```

```

    },
    { $unwind: "$movie" },
    {
      $project: {
        _id: 0,
        title: "$movie.title",
        ratingCount: 1,
        avgRating: { $round: ["$avgRating", 2] }
      }
    }
  ]
})

```

Output:

```

 mongosh mongodb://127.0.0.1:27017/cinematch?directConnection=true&serverSelectionTimeoutMS=2000

cinematch> db.ratings.aggregate([
  {
    $group: {
      _id: "$movieId",
      ratingCount: { $sum: 1 },
      avgRating: { $avg: "$rating" }
    }
  },
  { $sort: { ratingCount: -1 } },
  { $limit: 5 },
  {
    $lookup: {
      from: "movies",
      localField: "_id",
      foreignField: "movieId",
      as: "movie"
    }
  },
  { $unwind: "$movie" },
  {
    $project: {
      _id: 0,
      title: "$movie.title",
      ratingCount: 1,
      avgRating: { $round: ["$avgRating", 2] }
    }
  }
])
[
  { ratingCount: 329, title: 'Forrest Gump (1994)', avgRating: 4.16 },
  {
    ratingCount: 317,
    title: 'The Shawshank Redemption (1994)',
    avgRating: 4.43
  },
  { ratingCount: 307, title: 'Pulp Fiction (1994)', avgRating: 4.2 },
  {
    ratingCount: 279,
    title: 'The Silence of the Lambs (1991)',
    avgRating: 4.16
  },
  { ratingCount: 278, title: 'The Matrix (1999)', avgRating: 4.19 }
]
cinematch>

```

Result: Forrest Gump (329 ratings), The Shawshank Redemption (317), Pulp Fiction (307), The Silence of the Lambs (279) and The Matrix (278) are the most rated movies.

16. Top 5 highest-rated movies (minimum 100 ratings)

Query:

```
db.ratings.aggregate([
  {
    $group: {
      _id: "$movieId",
      ratingCount: { $sum: 1 },
      avgRating: { $avg: "$rating" }
    }
  },
  { $match: { ratingCount: { $gte: 100 } } },
  { $sort: { avgRating: -1 } },
  { $limit: 5 },
  {
    $lookup: {
      from: "movies",
      localField: "_id",
      foreignField: "movieId",
      as: "movie"
    }
  },
  { $unwind: "$movie" },
  {
    $project: {
      _id: 0,
      title: "$movie.title",
      year: "$movie.year",
      ratingCount: 1,
      avgRating: { $round: ["$avgRating", 2] }
    }
  }
])
```

Output:

```
mongosh mongodb://127.0.0.1:27017/cinematch?directConnection=true&serverSelectionTimeoutMS=2000

cinematch> db.ratings.aggregate([
  {
    $group: {
      _id: "$movieId",
      ratingCount: { $sum: 1 },
      avgRating: { $avg: "$rating" }
    }
  },
  { $match: { ratingCount: { $gte: 100 } } },
  { $sort: { avgRating: -1 } },
  { $limit: 5 },
  {
    $lookup: {
      from: "movies",
      localField: "_id",
      foreignField: "movieId",
      as: "movie"
    }
  },
  { $unwind: "$movie" },
  {
    $project: {
      _id: 0,
      title: "$movie.title",
      year: "$movie.year",
      ratingCount: 1,
      avgRating: { $round: ["$avgRating", 2] }
    }
  }
])
[
  {
    ratingCount: 317,
    title: 'The Shawshank Redemption (1994)',
    year: 1994,
    avgRating: 4.43
  },
  {
    ratingCount: 192,
    title: 'The Godfather (1972)',
    year: 1972,
    avgRating: 4.29
  },
  {
    ratingCount: 218,
    title: 'Fight Club (1999)',

```

```
mongosh mongodb://127.0.0.1:27017/cinematch?directConnection=true&serverSelectionTimeoutMS=2000

    as: "movie"
  }
},
{ $unwind: "$movie" },
{
  $project: {
    _id: 0,
    title: "$movie.title",
    year: "$movie.year",
    ratingCount: 1,
    avgRating: { $round: ["$avgRating", 2] }
  }
}
])
[
{
  ratingCount: 317,
  title: 'The Shawshank Redemption (1994)',
  year: 1994,
  avgRating: 4.43
},
{
  ratingCount: 192,
  title: 'The Godfather (1972)',
  year: 1972,
  avgRating: 4.29
},
{
  ratingCount: 218,
  title: 'Fight Club (1999)',
  year: 1999,
  avgRating: 4.27
},
{
  ratingCount: 129,
  title: 'The Godfather: Part II (1974)',
  year: 1974,
  avgRating: 4.26
},
{
  ratingCount: 107,
  title: 'The Departed (2006)',
  year: 2006,
  avgRating: 4.25
}
]
cinematch>
```

Result: Among movies with at least 100 ratings, The Shawshank Redemption (4.43) has the highest average, followed by The Godfather (4.29), Fight Club (4.27), The Godfather: Part II (4.26) and The Departed (4.25).

17. Most controversial movies (highest rating variance)

Query:

```
db.ratings.aggregate([
  {
    $group: {
      _id: "$movieId",
```

```

        ratingCount: { $sum: 1 },
        avgRating: { $avg: "$rating" },
        stdDev: { $stdDevPop: "$rating" }
    }
},
{ $match: { ratingCount: { $gte: 50 } } },
{ $sort: { stdDev: -1 } },
{ $limit: 5 },
{
    $lookup: {
        from: "movies",
        localField: "_id",
        foreignField: "movieId",
        as: "movie"
    }
},
{ $unwind: "$movie" },
{
    $project: {
        _id: 0,
        title: "$movie.title",
        ratingCount: 1,
        avgRating: { $round: ["$avgRating", 2] },
        stdDev: { $round: ["$stdDev", 3] }
    }
}
}
])

```

Output:

 mongosh mongodb://127.0.0.1:27017/cinematch?directConnection=true&serverSelectionTimeoutMS=2000

```
cinematch> db.ratings.aggregate([
  {
    $group: {
      _id: "$movieId",
      ratingCount: { $sum: 1 },
      avgRating: { $avg: "$rating" },
      stdDev: { $stdDevPop: "$rating" }
    }
  },
  { $match: { ratingCount: { $gte: 50 } } },
  { $sort: { stdDev: -1 } },
  { $limit: 5 },
  {
    $lookup: {
      from: "movies",
      localField: "_id",
      foreignField: "movieId",
      as: "movie"
    }
  },
  { $unwind: "$movie" },
  {
    $project: {
      _id: 0,
      title: "$movie.title",
      ratingCount: 1,
      avgRating: { $round: ["$avgRating", 2] },
      stdDev: { $round: ["$stdDev", 3] }
    }
  }
])
[
  {
    ratingCount: 64,
    title: 'The Blair Witch Project (1999)',
    avgRating: 2.8,
    stdDev: 1.363
  },
  {
    ratingCount: 54,
    title: 'First Knight (1995)',
    avgRating: 3.08,
    stdDev: 1.287
  },
  {
    ratingCount: 121,
```

```
mongosh mongodb://127.0.0.1:27017/cinematch?directConnection=true&serverSelectionTimeoutMS=2000

    as: "movie"
  }
},
{ $unwind: "$movie" },
{
  $project: {
    _id: 0,
    title: "$movie.title",
    ratingCount: 1,
    avgRating: { $round: ["$avgRating", 2] },
    stdDev: { $round: ["$stdDev", 3] }
  }
}
])
[
{
  ratingCount: 64,
  title: 'The Blair Witch Project (1999)',
  avgRating: 2.8,
  stdDev: 1.363
},
{
  ratingCount: 54,
  title: 'First Knight (1995)',
  avgRating: 3.08,
  stdDev: 1.287
},
{
  ratingCount: 121,
  title: 'Austin Powers: The Spy Who Shagged Me (1999)',
  avgRating: 3.2,
  stdDev: 1.21
},
{
  ratingCount: 70,
  title: 'Scream (1996)',
  avgRating: 3.2,
  stdDev: 1.187
},
{
  ratingCount: 55,
  title: 'Moulin Rouge (2001)',
  avgRating: 3.59,
  stdDev: 1.187
}
]
cinematch>
```

Result: The Blair Witch Project has the highest standard deviation (1.36), followed by First Knight and Austin Powers, which means users strongly disagree about them.

18. Weighted rating (IMDb-style Bayesian formula) – Top 5

Query:

```
db.ratings.aggregate([
  {
    $group: { _id: "$movieId", v: { $sum: 1 }, R: { $avg: "$rating" } }
  },
  { $match: { v: { $gte: 50 } } },
```

```

{
  $addFields: {
    weighted: {
      $add: [
        {
          $multiply: [{ $divide: ["$v", { $add: ["$v", 50] }] }, "$R"]
        },
        {
          $multiply: [{ $divide: [50, { $add: ["$v", 50] }] }, 3.5]
        }
      ]
    }
  },
  $sort: { weighted: -1 } },
{ $limit: 5 },
{
  $lookup: {
    from: "movies",
    localField: "_id",
    foreignField: "movieId",
    as: "movie"
  }
},
{ $unwind: "$movie" },
{
  $project: {
    _id: 0,
    title: "$movie.title",
    votes: "$v",
    avgRating: { $round: ["$R", 2] },
    weightedRating: { $round: ["$weighted", 3] }
  }
}
])

```

Output:

```
mongosh mongodb://127.0.0.1:27017/cinematch?directConnection=true&serverSelectionTimeoutMS=2000

cinematch> db.ratings.aggregate([
  {
    $group: { _id: "$movieId", v: { $sum: 1 }, R: { $avg: "$rating" } }
  },
  { $match: { v: { $gte: 50 } } },
  {
    $addFields: {
      weighted: {
        $add: [
          {
            $multiply: [{ $divide: ["$v", { $add: ["$v", 50] }] }, "$R"]
          },
          {
            $multiply: [{ $divide: [50, { $add: ["$v", 50] }] }, 3.5]
          }
        ]
      }
    }
  },
  { $sort: { weighted: -1 } },
  { $limit: 5 },
  {
    $lookup: {
      from: "movies",
      localField: "_id",
      foreignField: "movieId",
      as: "movie"
    }
  },
  { $unwind: "$movie" },
  {
    $project: {
      _id: 0,
      title: "$movie.title",
      votes: "$v",
      avgRating: { $round: ["$R", 2] },
      weightedRating: { $round: ["$weighted", 3] }
    }
  }
])

[
  {
    title: 'The Shawshank Redemption (1994)',
    votes: 317,
    avgRating: 4.43,
    weightedRating: 4.302
  }
]
```

```
mongosh mongodb://127.0.0.1:27017/cinematch?directConnection=true&serverSelectionTimeoutMS=2000

    as: "movie"
  }
},
{ $unwind: "$movie" },
{
  $project: {
    _id: 0,
    title: "$movie.title",
    votes: "$v",
    avgRating: { $round: ["$R", 2] },
    weightedRating: { $round: ["$weighted", 3] }
  }
}
])
[
{
  title: 'The Shawshank Redemption (1994)',
  votes: 317,
  avgRating: 4.43,
  weightedRating: 4.302
},
{
  title: 'Fight Club (1999)',
  votes: 218,
  avgRating: 4.27,
  weightedRating: 4.129
},
{
  title: 'The Godfather (1972)',
  votes: 192,
  avgRating: 4.29,
  weightedRating: 4.126
},
{
  title: 'Star Wars: Episode IV - A New Hope (1977)',
  votes: 251,
  avgRating: 4.23,
  weightedRating: 4.11
},
{
  title: 'Pulp Fiction (1994)',
  votes: 307,
  avgRating: 4.2,
  weightedRating: 4.099
}
]
cinematch>
```

Result: The weighted rating gives a balanced top 5 in which movies with very few ratings do not appear.

Genre Analysis

19. Average rating per genre

Query:

```
db.movies.aggregate([
  {
    $lookup: {
```

```

        from: "ratings",
        localField: "movieId",
        foreignField: "movieId",
        as: "r"
    }
},
{ $unwind: "$r" },
{ $unwind: "$genreList" },
{
    $group: {
        _id: "$genreList",
        avgRating: { $avg: "$r.rating" },
        ratings: { $sum: 1 }
    }
},
{
    $project: { avgRating: { $round: ["$avgRating", 3] }, ratings: 1 }
},
{ $sort: { avgRating: -1 } }
])

```

Output:

```

mongosh mongodb://127.0.0.1:27017/cinematch?directConnection=true&serverSelectionTimeoutMS=2000

cinematch> db.movies.aggregate([
|   {
|     $lookup: {
|       from: "ratings",
|       localField: "movieId",
|       foreignField: "movieId",
|       as: "r"
|     }
|   },
|   { $unwind: "$r" },
|   { $unwind: "$genreList" },
|   {
|     $group: {
|       _id: "$genreList",
|       avgRating: { $avg: "$r.rating" },
|       ratings: { $sum: 1 }
|     }
|   },
|   {
|     $project: { avgRating: { $round: ["$avgRating", 3] }, ratings: 1 }
|   },
|   { $sort: { avgRating: -1 } }
| ])

```

Result: Film-Noir (3.92), War (3.81) and Documentary (3.80) have the highest average rating. Horror (3.26) has the lowest.

20. Best movie in each genre (minimum 50 ratings, first 5 genres)

Query:

```

db.ratings.aggregate([
  {

```

```

    $group: {
      _id: "$movieId",
      ratingCount: { $sum: 1 },
      avgRating: { $avg: "$rating" }
    }
  },
  { $match: { ratingCount: { $gte: 50 } } },
  {
    $lookup: {
      from: "movies",
      localField: "_id",
      foreignField: "movieId",
      as: "movie"
    }
  },
  { $unwind: "$movie" },
  { $unwind: "$movie.genreList" },
  { $sort: { avgRating: -1 } },
  {
    $group: {
      _id: "$movie.genreList",
      bestMovie: { $first: "$movie.title" },
      avgRating: { $first: { $round: ["$avgRating", 2] } },
      ratingCount: { $first: "$ratingCount" }
    }
  },
  { $sort: { _id: 1 } },
  { $limit: 5 }
])

```

Output:

```
mongosh mongodb://127.0.0.1:27017/cinematch?directConnection=true&serverSelectionTimeoutMS=2000

cinematch> db.ratings.aggregate([
  {
    $group: {
      _id: "$movieId",
      ratingCount: { $sum: 1 },
      avgRating: { $avg: "$rating" }
    }
  },
  { $match: { ratingCount: { $gte: 50 } } },
  {
    $lookup: {
      from: "movies",
      localField: "_id",
      foreignField: "movieId",
      as: "movie"
    }
  },
  { $unwind: "$movie" },
  { $unwind: "$movie.genreList" },
  { $sort: { avgRating: -1 } },
  {
    $group: {
      _id: "$movie.genreList",
      bestMovie: { $first: "$movie.title" },
      avgRating: { $first: { $round: ["$avgRating", 2] } },
      ratingCount: { $first: "$ratingCount" }
    }
  },
  { $sort: { _id: 1 } },
  { $limit: 5 }
])
[
  {
    _id: 'Action',
    bestMovie: 'Fight Club (1999)',
    avgRating: 4.27,
    ratingCount: 218
  },
  {
    _id: 'Adventure',
    bestMovie: 'The Princess Bride (1987)',
    avgRating: 4.23,
    ratingCount: 142
  },
  {
    _id: 'Animation',

```

```
mongosh mongodb://127.0.0.1:27017/cinematch?directConnection=true&serverSelectionTimeoutMS=2000
{ $unwind: "$movie" },
{ $unwind: "$movie.genreList" },
{ $sort: { avgRating: -1 } },
{
  $group: {
    _id: "$movie.genreList",
    bestMovie: { $first: "$movie.title" },
    avgRating: { $first: { $round: ["$avgRating", 2] } },
    ratingCount: { $first: "$ratingCount" }
  }
},
{ $sort: { _id: 1 } },
{ $limit: 5 }
])
[
{
  _id: 'Action',
  bestMovie: 'Fight Club (1999)',
  avgRating: 4.27,
  ratingCount: 218
},
{
  _id: 'Adventure',
  bestMovie: 'The Princess Bride (1987)',
  avgRating: 4.23,
  ratingCount: 142
},
{
  _id: 'Animation',
  bestMovie: 'Spirited Away (Sen to Chihiro no kamikakushi) (2001)',
  avgRating: 4.16,
  ratingCount: 87
},
{
  _id: 'Children',
  bestMovie: 'Toy Story 3 (2010)',
  avgRating: 4.11,
  ratingCount: 55
},
{
  _id: 'Comedy',
  bestMovie: 'Dr. Strangelove or: How I Learned to Stop Worrying and Love the Bomb (1964)',
  avgRating: 4.27,
  ratingCount: 97
}
]
cinematch>
```

Result: The best rated movie of the first five genres is shown, for example Fight Club for Action and Spirited Away for Animation.

User Analysis

21. Top 5 most active users

Query:

```
db.ratings.aggregate([
  {
    $group: {
      _id: "$userId",
      ratings: { $sum: 1 },
      avgRating: { $avg: "$rating" }
    }
  }
])
```

```

    }
  },
  { $sort: { ratings: -1 } },
  { $limit: 5 },
  {
    $project: {
      userId: "$_id",
      _id: 0,
      ratings: 1,
      avgRating: { $round: ["$avgRating", 2] }
    }
  }
]
)

```

Output:

```

C:\> mongosh mongodb://127.0.0.1:27017/cinematch?directConnection=true&serverSelectionTimeoutMS=2000

cinematch> db.ratings.aggregate([
  {
    $group: {
      _id: "$userId",
      ratings: { $sum: 1 },
      avgRating: { $avg: "$rating" }
    }
  },
  { $sort: { ratings: -1 } },
  { $limit: 5 },
  {
    $project: {
      userId: "$_id",
      _id: 0,
      ratings: 1,
      avgRating: { $round: ["$avgRating", 2] }
    }
  }
])
[
  { ratings: 2698, userId: 414, avgRating: 3.39 },
  { ratings: 2478, userId: 599, avgRating: 2.64 },
  { ratings: 2108, userId: 474, avgRating: 3.4 },
  { ratings: 1864, userId: 448, avgRating: 2.85 },
  { ratings: 1346, userId: 274, avgRating: 3.24 }
]
cinematch>

```

Result: User 414 has given the most ratings (2,698), followed by user 599 (2,478), user 474 (2,108), user 448 (1,864) and user 274 (1,346).

22. Harshest and most generous critics (minimum 100 ratings)

Query:

```

db.ratings.aggregate([
  {
    $group: {
      _id: "$userId",
      ratings: { $sum: 1 },
      avgRating: { $avg: "$rating" }
    }
  }
])

```

```
},
{ $match: { ratings: { $gte: 100 } } },
{
  $project: {
    userId: "$_id",
    _id: 0,
    ratings: 1,
    avgRating: { $round: ["$avgRating", 2] }
  }
},
{
  $facet: {
    harshest: [{ $sort: { avgRating: 1 } }, { $limit: 5 }],
    mostGenerous: [{ $sort: { avgRating: -1 } }, { $limit: 5 } ]
  }
}
])
```

Output:

```
mongosh mongodb://127.0.0.1:27017/cinematch?directConnection=true&serverSelectionTimeoutMS=2000

cinematch> db.ratings.aggregate([
  {
    $group: {
      _id: "$userId",
      ratings: { $sum: 1 },
      avgRating: { $avg: "$rating" }
    }
  },
  { $match: { ratings: { $gte: 100 } } },
  {
    $project: {
      userId: "$_id",
      _id: 0,
      ratings: 1,
      avgRating: { $round: ["$avgRating", 2] }
    }
  },
  {
    $facet: {
      harshest: [{ $sort: { avgRating: 1 } }, { $limit: 5 }],
      mostGenerous: [{ $sort: { avgRating: -1 } }, { $limit: 5 } ]
    }
  }
])

[
  {
    harshest: [
      { ratings: 194, userId: 139, avgRating: 2.14 },
      { ratings: 179, userId: 153, avgRating: 2.22 },
      { ratings: 385, userId: 567, avgRating: 2.25 },
      { ratings: 939, userId: 298, avgRating: 2.36 },
      { ratings: 400, userId: 517, avgRating: 2.39 }
    ],
    mostGenerous: [
      { ratings: 202, userId: 452, avgRating: 4.56 },
      { ratings: 292, userId: 122, avgRating: 4.55 },
      { ratings: 114, userId: 43, avgRating: 4.55 },
      { ratings: 130, userId: 52, avgRating: 4.48 },
      { ratings: 101, userId: 601, avgRating: 4.43 }
    ]
  }
]
cinematch>
```

Result: User 139 gives the lowest average rating (2.14) and user 452 gives the highest (4.56). Both lists are returned by one \$facet query.

23. Distribution of ratings per user

Query:

```
db.ratings.aggregate([
  { $group: { _id: "$userId", ratings: { $sum: 1 } } },
  {
    $bucket: {
      groupBy: "$ratings",
      boundaries: [20, 50, 100, 200, 500, 1000, 3000],
```

```

        default: "3000+",
        output: { users: { $sum: 1 } } }
    }
}
])

```

Output:

```

C:\> mongosh mongodb://127.0.0.1:27017/cinematch?directConnection=true&serverSelectionTimeoutMS=2000

cinematch> db.ratings.aggregate([
|   { $group: { _id: "$userId", ratings: { $sum: 1 } } },
|   {
|     $bucket: {
|       groupBy: "$ratings",
|       boundaries: [20, 50, 100, 200, 500, 1000, 3000],
|       default: "3000+",
|       output: { users: { $sum: 1 } }
|     }
|   }
| ])
|
| [
|   { _id: 20, users: 225 },
|   { _id: 50, users: 137 },
|   { _id: 100, users: 114 },
|   { _id: 200, users: 90 },
|   { _id: 500, users: 32 },
|   { _id: 1000, users: 12 }
| ]
cinematch>

```

Result: Most users have between 20 and 99 ratings. Only 12 users have more than 1,000 ratings.

24. Number of ratings submitted per year

Query:

```

db.ratings.aggregate([
  {
    $group: {
      _id: { $year: "$ratedAt" },
      ratings: { $sum: 1 },
      avgRating: { $avg: "$rating" }
    }
  },
  {
    $project: { ratings: 1, avgRating: { $round: ["$avgRating", 2] } }
  },
  { $sort: { _id: 1 } }
]).toArray()

```

Output:

```
mongosh mongodb://127.0.0.1:27017/cinematch?directConnection=true&serverSelectionTimeoutMS=2000

cinematch> db.ratings.aggregate([
  {
    $group: {
      _id: { $year: "$ratedAt" },
      ratings: { $sum: 1 },
      avgRating: { $avg: "$rating" }
    }
  },
  {
    $project: { ratings: 1, avgRating: { $round: ["$avgRating", 2] } }
  },
  { $sort: { _id: 1 } }
]).toArray()
[
  { _id: 1996, ratings: 6040, avgRating: 3.54 },
  { _id: 1997, ratings: 1916, avgRating: 3.73 },
  { _id: 1998, ratings: 507, avgRating: 3.44 },
  { _id: 1999, ratings: 2439, avgRating: 3.77 },
  { _id: 2000, ratings: 10061, avgRating: 3.39 },
  { _id: 2001, ratings: 3922, avgRating: 3.51 },
  { _id: 2002, ratings: 3478, avgRating: 3.61 },
  { _id: 2003, ratings: 4014, avgRating: 3.5 },
  { _id: 2004, ratings: 3279, avgRating: 3.51 },
  { _id: 2005, ratings: 5813, avgRating: 3.43 },
  { _id: 2006, ratings: 4059, avgRating: 3.49 },
  { _id: 2007, ratings: 7114, avgRating: 3.31 },
  { _id: 2008, ratings: 4351, avgRating: 3.66 },
  { _id: 2009, ratings: 4158, avgRating: 3.39 },
  { _id: 2010, ratings: 2301, avgRating: 3.46 },
  { _id: 2011, ratings: 1690, avgRating: 3.78 },
  { _id: 2012, ratings: 4656, avgRating: 3.64 },
  { _id: 2013, ratings: 1664, avgRating: 3.88 },
  { _id: 2014, ratings: 1439, avgRating: 3.51 },
  { _id: 2015, ratings: 6616, avgRating: 3.51 },
  { _id: 2016, ratings: 6703, avgRating: 3.69 },
  { _id: 2017, ratings: 8198, avgRating: 3.35 },
  { _id: 2018, ratings: 6418, avgRating: 3.39 }
]
cinematch> _
```

Result: The most ratings were given in the year 2000 (10,061), followed by 2007 and 2015.

Tag Analysis

25. Most frequently used tags

Query:

```
db.tags.aggregate([
  { $group: { _id: { $toLower: "$tag" }, count: { $sum: 1 } } },
  { $sort: { count: -1 } },
  { $limit: 5 }
])
```

Output:

```
mongosh mongodb://127.0.0.1:27017/cinematch?directConnection=true&serverSelectionTimeoutMS=2000

cinematch> db.tags.aggregate([
|   { $group: { _id: { $toLowerCase: "$tag" }, count: { $sum: 1 } } },
|   { $sort: { count: -1 } },
|   { $limit: 5 }
| ])
[
|   { _id: 'in netflix queue', count: 131 },
|   { _id: 'atmospheric', count: 41 },
|   { _id: 'funny', count: 24 },
|   { _id: 'superhero', count: 24 },
|   { _id: 'thought-provoking', count: 24 }
| ]
cinematch>
```

Result: 'in netflix queue' is the most used tag (131 times), followed by 'atmospheric' (41).

26. Most tagged movies

Query:

```
db.tags.aggregate([
  { $group: { _id: "$movieId", tagCount: { $sum: 1 } } },
  { $sort: { tagCount: -1 } },
  { $limit: 5 },
  {
    $lookup: {
      from: "movies",
      localField: "_id",
      foreignField: "movieId",
      as: "movie"
    }
  },
  { $unwind: "$movie" },
  { $project: { _id: 0, title: "$movie.title", tagCount: 1 } }
])
```

Output:

```
mongosh mongodb://127.0.0.1:27017/cinematch?directConnection=true&serverSelectionTimeoutMS=2000

cinematch> db.tags.aggregate([
  { $group: { _id: "$movieId", tagCount: { $sum: 1 } } },
  { $sort: { tagCount: -1 } },
  { $limit: 5 },
  {
    $lookup: {
      from: "movies",
      localField: "_id",
      foreignField: "movieId",
      as: "movie"
    }
  },
  { $unwind: "$movie" },
  { $project: { _id: 0, title: "$movie.title", tagCount: 1 } }
])
[
  { tagCount: 181, title: 'Pulp Fiction (1994)' },
  { tagCount: 54, title: 'Fight Club (1999)' },
  { tagCount: 41, title: '2001: A Space Odyssey (1968)' },
  {
    tagCount: 35,
    title: 'Léon: The Professional (a.k.a. The Professional) (Léon) (1994)'
  },
  { tagCount: 34, title: 'Eternal Sunshine of the Spotless Mind (2004)' }
]
cinematch> _
```

Result: Pulp Fiction (181) and Fight Club (54) are the most tagged movies, followed by 2001: A Space Odyssey, Leon and Eternal Sunshine of the Spotless Mind.

Recommendation Queries

27. Content-based: movies most similar to Toy Story by genre overlap

Query:

```
db.movies.aggregate([
  { $match: { movieId: { $ne: 1 } } },
  {
    $addFields: {
      sharedGenres: {
        $size: {
          $setIntersection: [
            "$genreList",
            [
              "Adventure",
              "Animation",
              "Children",
              "Comedy",
              "Fantasy"
            ]
          ]
        }
      }
    }
  }
])
```

```

    },
    { $match: { sharedGenres: { $gte: 4 } } } },
    {
      $lookup: {
        from: "ratings",
        localField: "movieId",
        foreignField: "movieId",
        as: "r"
      }
    },
    {
      $addFields: {
        ratingCount: { $size: "$r" },
        avgRating: { $avg: "$r.rating" }
      }
    },
    { $match: { ratingCount: { $gte: 20 } } } },
    { $sort: { sharedGenres: -1, avgRating: -1 } } },
    { $limit: 5 },
    {
      $project: {
        _id: 0,
        title: 1,
        sharedGenres: 1,
        ratingCount: 1,
        avgRating: { $round: ["$avgRating", 2] }
      }
    }
  ]
}
]
)

```

Output:

 mongosh mongodb://127.0.0.1:27017/cinematch?directConnection=true&serverSelectionTimeoutMS=2000

```
cinematch> db.movies.aggregate([
  { $match: { movieId: { $ne: 1 } } },
  {
    $addFields: {
      sharedGenres: {
        $size: {
          $setIntersection: [
            "$genreList",
            [
              "Adventure",
              "Animation",
              "Children",
              "Comedy",
              "Fantasy"
            ]
          ]
        }
      }
    }
  },
  { $match: { sharedGenres: { $gte: 4 } } },
  {
    $lookup: {
      from: "ratings",
      localField: "movieId",
      foreignField: "movieId",
      as: "r"
    }
  },
  {
    $addFields: {
      ratingCount: { $size: "$r" },
      avgRating: { $avg: "$r.rating" }
    }
  },
  { $match: { ratingCount: { $gte: 20 } } },
  { $sort: { sharedGenres: -1, avgRating: -1 } },
  { $limit: 5 },
  {
    $project: {
      _id: 0,
      title: 1,
      sharedGenres: 1,
      ratingCount: 1,
      avgRating: { $round: ["$avgRating", 2] }
    }
  }
])
```

```
mongosh mongodb://127.0.0.1:27017/cinematch?directConnection=true&serverSelectionTimeoutMS=2000
[
  {
    $match: { ratingCount: { $gte: 20 } } },
    { $sort: { sharedGenres: -1, avgRating: -1 } },
    { $limit: 5 },
    {
      $project: {
        _id: 0,
        title: 1,
        sharedGenres: 1,
        ratingCount: 1,
        avgRating: { $round: ["$avgRating", 2] }
      }
    }
  ]
)
[
  {
    title: 'Toy Story 3 (2010)',
    sharedGenres: 5,
    ratingCount: 55,
    avgRating: 4.11
  },
  {
    title: 'Monsters, Inc. (2001)',
    sharedGenres: 5,
    ratingCount: 132,
    avgRating: 3.87
  },
  {
    title: 'The Lego Movie (2014)',
    sharedGenres: 5,
    ratingCount: 31,
    avgRating: 3.87
  },
  {
    title: 'Shrek (2001)',
    sharedGenres: 5,
    ratingCount: 170,
    avgRating: 3.87
  },
  {
    title: 'Toy Story 2 (1999)',
    sharedGenres: 5,
    ratingCount: 97,
    avgRating: 3.86
  }
]
cinematch> _
```

Result: Toy Story 3, Monsters Inc., The Lego Movie, Shrek and Toy Story 2 share all five genres with Toy Story and are recommended.

28. Collaborative: 'users who liked The Matrix also liked...'

Query:

```
db.ratings.aggregate([
  { $match: { movieId: 2571, rating: { $gte: 4.5 } } },
  { $group: { _id: null, fans: { $addToSet: "$userId" } } },
  {
    $lookup: {
```

```

    from: "ratings",
    let: { fans: "$fans" },
    pipeline: [
      {
        $match: {
          $expr: {
            $and: [
              { $in: ["$userId", "$$fans"] },
              { $gte: ["$rating", 4.5] },
              { $ne: ["$movieId", 2571] }
            ]
          }
        },
        { $group: { _id: "$movieId", fanCount: { $sum: 1 } } },
        { $sort: { fanCount: -1 } },
        { $limit: 5 }
      ],
      as: "alsoLiked"
    ]
  },
  { $unwind: "$alsoLiked" },
  {
    $lookup: {
      from: "movies",
      localField: "alsoLiked._id",
      foreignField: "movieId",
      as: "movie"
    }
  },
  {
    $project: {
      _id: 0,
      title: { $first: "$movie.title" },
      fansWhoLikedIt: "$alsoLiked.fanCount",
      totalMatrixFans: { $size: "$fans" }
    }
  }
])

```

Output:

 mongosh mongodb://127.0.0.1:27017/cinematch?directConnection=true&serverSelectionTimeoutMS=2000

```
cinematch> db.ratings.aggregate([
|   { $match: { movieId: 2571, rating: { $gte: 4.5 } } },
|   { $group: { _id: null, fans: { $addToSet: "$userId" } } },
|   {
|     $lookup: {
|       from: "ratings",
|       let: { fans: "$fans" },
|       pipeline: [
|         {
|           $match: {
|             $expr: {
|               $and: [
|                 { $in: ["$userId", "$$fans"] },
|                 { $gte: ["$rating", 4.5] },
|                 { $ne: ["$movieId", 2571] }
|               ]
|             }
|           }
|         },
|         { $group: { _id: "$movieId", fanCount: { $sum: 1 } } },
|         { $sort: { fanCount: -1 } },
|         { $limit: 5 }
|       ],
|       as: "alsoLiked"
|     }
|   },
|   { $unwind: "$alsoLiked" },
|   {
|     $lookup: {
|       from: "movies",
|       localField: "alsoLiked._id",
|       foreignField: "movieId",
|       as: "movie"
|     }
|   },
|   {
|     $project: {
|       _id: 0,
|       title: { $first: "$movie.title" },
|       fansWhoLikedIt: "$alsoLiked.fanCount",
|       totalMatrixFans: { $size: "$fans" }
|     }
|   }
| ])
| {
```

```
mongosh mongodb://127.0.0.1:27017/cinematch?directConnection=true&serverSelectionTimeoutMS=2000
[
  {
    { $unwind: "$alsoLiked" },
    {
      $lookup: {
        from: "movies",
        localField: "alsoLiked._id",
        foreignField: "movieId",
        as: "movie"
      }
    },
    {
      $project: {
        _id: 0,
        title: { $first: "$movie.title" },
        fansWhoLikedIt: "$alsoLiked.fanCount",
        totalMatrixFans: { $size: "$fans" }
      }
    }
  }
])
[
  {
    title: 'The Shawshank Redemption (1994)',
    fansWhoLikedIt: 75,
    totalMatrixFans: 150
  },
  {
    title: 'Fight Club (1999)',
    fansWhoLikedIt: 75,
    totalMatrixFans: 150
  },
  {
    title: 'Pulp Fiction (1994)',
    fansWhoLikedIt: 61,
    totalMatrixFans: 150
  },
  {
    title: 'Star Wars: Episode IV - A New Hope (1977)',
    fansWhoLikedIt: 60,
    totalMatrixFans: 150
  },
  {
    title: 'Star Wars: Episode V - The Empire Strikes Back (1980)',
    fansWhoLikedIt: 59,
    totalMatrixFans: 150
  }
]
cinematch> _
```

Result: Out of 150 users who gave The Matrix 4.5 or 5 stars, 75 also gave high ratings to The Shawshank Redemption and Fight Club, and 61 to Pulp Fiction.

29. Personalised recommendations for user 1

Query:

```
db.movies.aggregate([
  { $match: { genreList: { $all: ["Action", "Adventure"] } } },
  {
    $lookup: {
      from: "ratings",
```

```

        localField: "movieId",
        foreignField: "movieId",
        as: "r"
    }
},
{ $match: { "r.userId": { $ne: 1 } } },
{
    $addFields: {
        ratingCount: { $size: "$r" },
        avgRating: { $avg: "$r.rating" }
    }
},
{ $match: { ratingCount: { $gte: 50 } } },
{ $sort: { avgRating: -1 } },
{ $limit: 5 },
{
    $project: {
        _id: 0,
        title: 1,
        year: 1,
        ratingCount: 1,
        avgRating: { $round: ["$avgRating", 2] }
    }
}
})

```

Output:

```
mongosh mongodb://127.0.0.1:27017/cinamatch?directConnection=true&serverSelectionTimeoutMS=2000

cinematch> db.movies.aggregate([
  { $match: { genreList: { $all: ["Action", "Adventure"] } } },
  {
    $lookup: {
      from: "ratings",
      localField: "movieId",
      foreignField: "movieId",
      as: "r"
    }
  },
  { $match: { "r.userId": { $ne: 1 } } },
  {
    $addFields: {
      ratingCount: { $size: "$r" },
      avgRating: { $avg: "$r.rating" }
    }
  },
  { $match: { ratingCount: { $gte: 50 } } },
  { $sort: { avgRating: -1 } },
  { $limit: 5 },
  {
    $project: {
      _id: 0,
      title: 1,
      year: 1,
      ratingCount: 1,
      avgRating: { $round: ["$avgRating", 2] }
    }
  }
])
[
  {
    title: 'North by Northwest (1959)',
    year: 1959,
    ratingCount: 57,
    avgRating: 4.18
  },
  {
    title: 'City of God (Cidade de Deus) (2002)',
    year: 2002,
    ratingCount: 75,
    avgRating: 4.15
  },
  {
    title: 'The Good, the Bad and the Ugly (Buono, il brutto, il cattivo, Il) (1966)',
    year: 1966,
```

```
mongosh mongodb://127.0.0.1:27017/cinematch?directConnection=true&serverSelectionTimeoutMS=2000
[
  {
    $match: { ratingCount: { $gte: 50 } },
    $sort: { avgRating: -1 },
    $limit: 5,
    {
      $project: {
        _id: 0,
        title: 1,
        year: 1,
        ratingCount: 1,
        avgRating: { $round: ["$avgRating", 2] }
      }
    }
  ]
})
[
  {
    title: 'North by Northwest (1959)',
    year: 1959,
    ratingCount: 57,
    avgRating: 4.18
  },
  {
    title: 'City of God (Cidade de Deus) (2002)',
    year: 2002,
    ratingCount: 75,
    avgRating: 4.15
  },
  {
    title: 'The Good, the Bad and the Ugly (Buono, il brutto, il cattivo, Il) (1966)',
    year: 1966,
    ratingCount: 72,
    avgRating: 4.15
  },
  {
    title: 'The Lord of the Rings: The Return of the King (2003)',
    year: 2003,
    ratingCount: 185,
    avgRating: 4.12
  },
  {
    title: 'Guardians of the Galaxy (2014)',
    year: 2014,
    ratingCount: 59,
    avgRating: 4.05
  }
]
cinematch> 
```

Result: Action and Adventure movies that user 1 has not rated are recommended, such as North by Northwest, City of God and The Good, the Bad and the Ugly.

CRUD & Index Performance

30. Explain plan: index scan vs collection scan

Query:

```
const plan = db.ratings.find({ movieId: 296 }).explain("executionStats");
({
  stage: plan.queryPlanner.winningPlan.stage,
```

```

    inputStage: plan.queryPlanner.winningPlan.inputStage?.stage,
    indexName: plan.queryPlanner.winningPlan.inputStage?.indexName,
    docsReturned: plan.executionStats.nReturned,
    docsExamined: plan.executionStats.totalDocsExamined,
    keysExamined: plan.executionStats.totalKeysExamined,
    executionTimeMs: plan.executionStats.executionTimeMillis
  })
})

```

Output:

```

C:\> mongosh mongodb://127.0.0.1:27017/cinematch?directConnection=true&serverSelectionTimeoutMS=2000

cinematch> const plan = db.ratings.find({ movieId: 296 }).explain("executionStats");

cinematch> ({
  stage: plan.queryPlanner.winningPlan.stage,
  inputStage: plan.queryPlanner.winningPlan.inputStage?.stage,
  indexName: plan.queryPlanner.winningPlan.inputStage?.indexName,
  docsReturned: plan.executionStats.nReturned,
  docsExamined: plan.executionStats.totalDocsExamined,
  keysExamined: plan.executionStats.totalKeysExamined,
  executionTimeMs: plan.executionStats.executionTimeMillis
})
{
  stage: 'FETCH',
  inputStage: 'IXSCAN',
  indexName: 'movieId_1',
  docsReturned: 307,
  docsExamined: 307,
  keysExamined: 307,
  executionTimeMs: 0
}
cinematch> 

```

Result: The query uses the movieId index (IXSCAN). Only 307 documents were examined instead of all 100,836.

MongoDB Operators Used

Operator	Use in this project
find, findOne, countDocuments, distinct	Reading documents, counting and unique values
\$regex, \$text	Pattern matching and text search on titles
\$all, \$in, \$ne, \$gte, \$lte, \$size, \$expr	Filtering conditions on fields and arrays
updateMany with \$set	Adding new fields to existing documents
\$match	Filters documents in the pipeline
\$group with \$sum, \$avg, \$min, \$max, \$first, \$addToSet, \$stdDevPop	Grouping and calculating values
\$sort, \$limit, \$count, \$project, \$addFields	Sorting, limiting and shaping the output
\$lookup	Joining information from another collection
\$unwind	Expanding an array into separate documents
\$facet, \$bucket	Multiple results in one query, grouping into ranges
\$split, \$regexFind, \$concat, \$toLower, \$toInt, \$toDate	String and type conversion
\$year, \$dayOfWeek	Date parts
\$setIntersection, \$arrayElemAt, \$slice	Array operations
createIndex, getIndexes, explain	Indexing

CineMatch Web Application

A web application was developed so that the analysis can be viewed without opening mongosh. The backend is written in Node.js with Express and connects to MongoDB using the official driver. It has API endpoints for the overview statistics, movie search, movie details with recommendations, genre analysis, user profiles and for running any of the queries above. The frontend is written in React and shows the results as tables and charts.

Endpoint	Description
GET /api/overview	Counts, rating distribution, ratings per year, genre statistics, top movies
GET /api/movies	Search movies by title, genre and sort order
GET /api/movies/:id	Movie details, rating breakdown, tags and recommendations
GET /api/genres/:genre	Movies per decade and top movies of a genre
GET /api/users/:id	User profile, favourite genres and recommended movies
GET /api/queries/:id/run	Runs one of the queries of this report



Overview page


CineMatch
MONGODB
ANALYTICS

Overview

Explore movies

Genres

Users & recs

Query Lab

EXPLORE

Movie catalogue

Regex title search, multkey-index genre filters and sorting on the pre-aggregated movie_stats collection.

Search

Highest rated

≥ 100 ratings

All genres

Action

Adventure

Animation

Children

Comedy

Crime

Documentary

Drama

Fantasy

Film-Noir

Horror

IMAX

Musical

Mystery

Romance

Sci-Fi

Thriller

War

Western

138 movies

◀ Prev

page 1 / 6

Next ▶

#1

The Shawshank Redemption

1994

★ 4.43

317

Crime

Drama

#2

The Godfather

1972

★ 4.29

192

Crime

Drama

#3

Fight Club

1999

★ 4.27

218

Action

Crime

Drama

#4

The Godfather: Part II

1974

★ 4.26

129

Crime

Drama

#5

The Departed

2006

★ 4.25

107

Crime

Drama

Thriller

#6

Goodfellas

1990

★ 4.25

126

Crime

Drama

#7

Casablanca

1942

★ 4.24

100

Drama

Romance

#8

The Dark Knight

2008

★ 4.24

149

Action

Crime

Drama

#9

The Usual Suspects

1995

★ 4.24

204

Crime

Mystery

Thriller

#10

The Princess Bride

1987

★ 4.23

142

Action

Adventure

Comedy

#11

Star Wars: Episode IV - A New Hope

1977

★ 4.23

251

Action

Adventure

Sci-Fi

#12

Schindler's List

1993

★ 4.22

220

Drama

War

#13

Apocalypse Now

1979

★ 4.22

107

Action

Drama

War

#14

American History X

1998

★ 4.22

129

Crime

Drama

#15

Star Wars: Episode V - The Empire Strikes Back

1980

★ 4.22

211

Action

Adventure

Sci-Fi

#16

Raiders of the Lost Ark (Indiana Jones and the Raiders of the Lost Ark)

1981

★ 4.21

200

Action

Adventure

#17

One Flew Over the Cuckoo's Nest

1975

★ 4.20

133

Drama

#18

Reservoir Dogs

1992

★ 4.20

131

Crime

Mystery

Thriller

#19

Pulp Fiction

1994

★ 4.20

307

Comedy

Crime

Drama

#20

The Matrix

1999

★ 4.19

278

Action

Sci-Fi

Thriller

#21

#22

#23

#24

Database cinematch

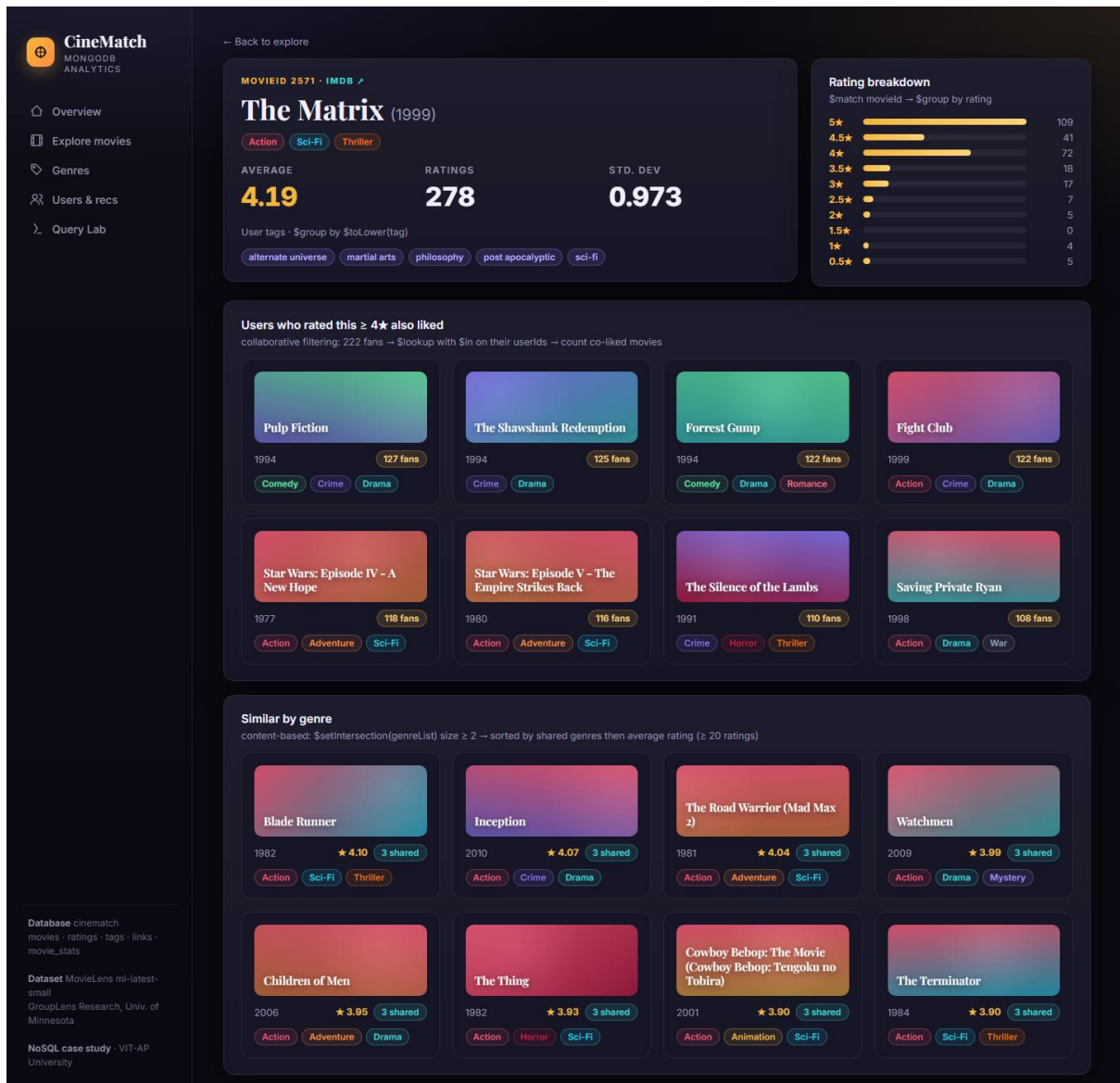
movies · ratings · tags · links · movie_stats

Dataset MovieLens ml-latest-small

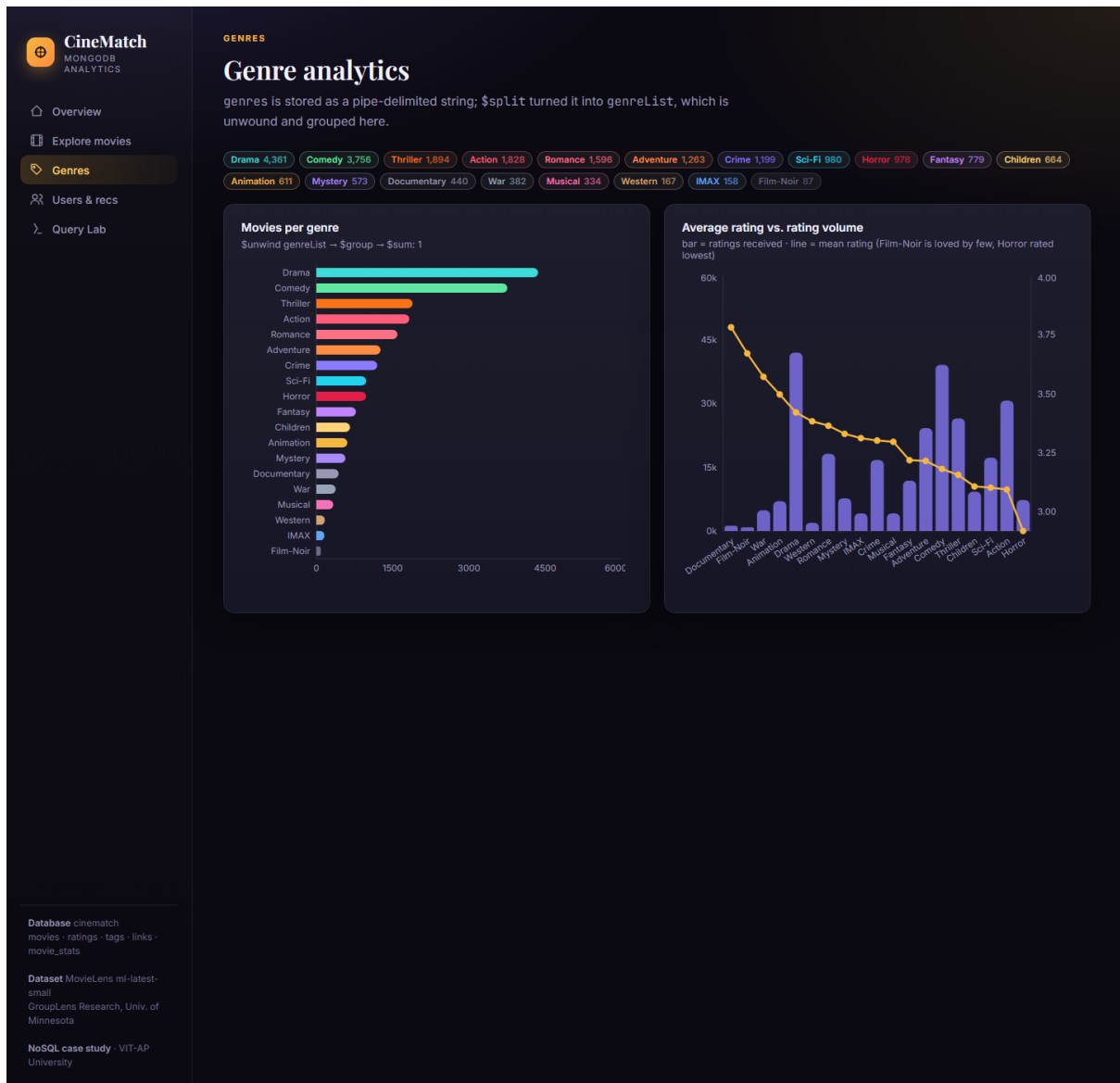
GroupLens Research, Univ. of Minnesota

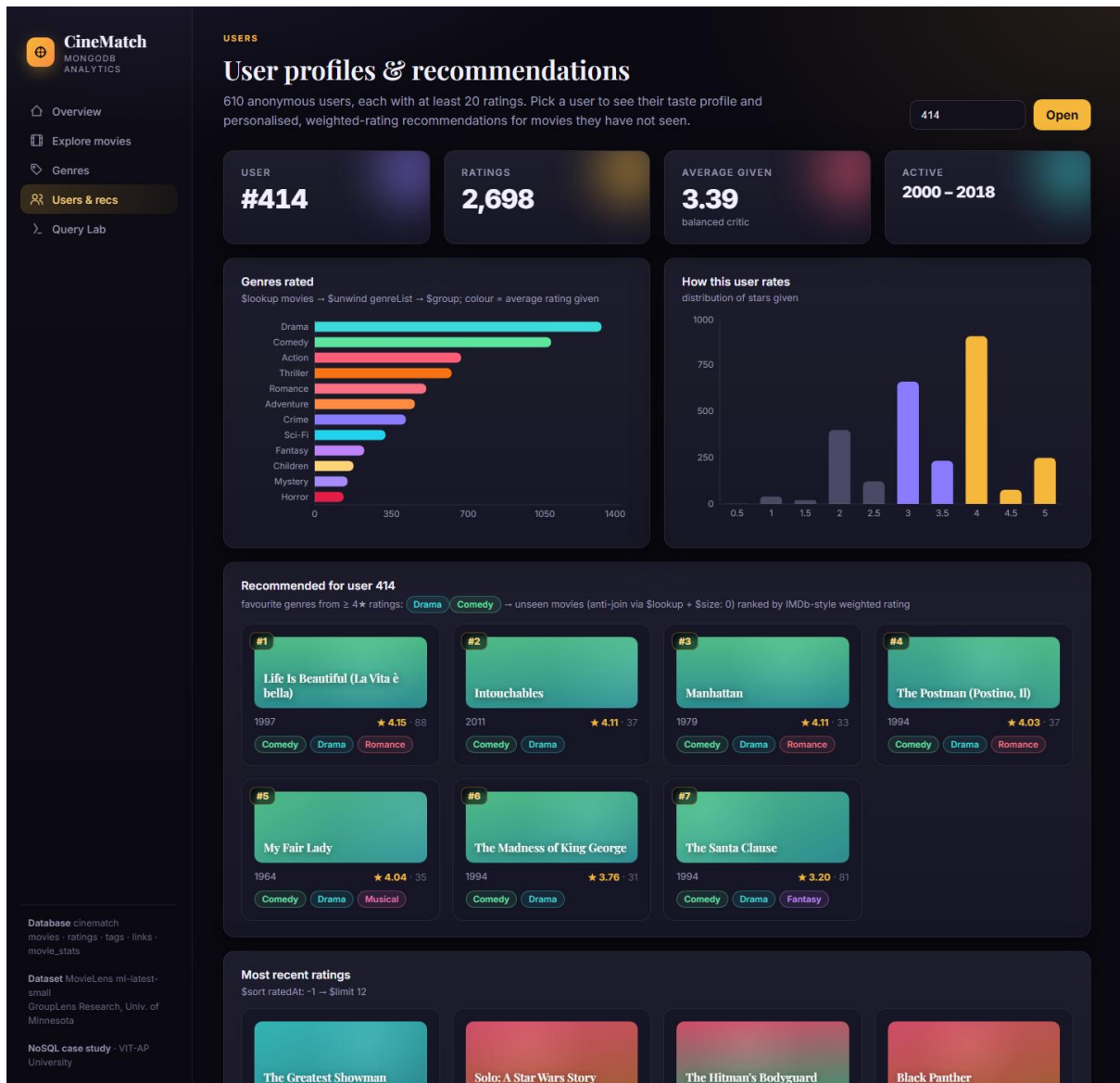
NoSQL case study · VIT-AP University

Explore page - highest rated movies with at least 100 ratings



Movie page of The Matrix with recommendations






CineMatch
MONGODB
ANALYTICS

Overview
Explore movies
Genres
Users & recs
Query Lab

Database: cinematch
collections: movies, ratings, tags, links, movie_stats
Dataset: MovieLens ml-latest-small
GroupLens Research, Univ. of Minnesota
NoSQL case study - VIT-AP University

QUERY LAB

All 57 MongoDB queries

Every query from the case study, exactly as typed in mongosh. Select one to execute it live against the cinematch database and see the result as a table, chart or raw JSON.

DATABASE OVERVIEW

- List all collections in the cinematch database
- Count documents in every collection
- Display a sample movie document
- Display a sample rating document
- Display a sample tag and link document
- Count the number of distinct users who rated movies

DATA CLEANING & SCHEMA ENRICHMENT

- Extract the release year from the movie title
- Split the pipe-delimited genres string into an array
- Convert Unix timestamps into ISODate fields
- Verify the enriched movie document
- Create indexes on frequently queried fields

MOVIE EXPLORATION

- Find movies released in 1995
- Search movie titles with a regular expression
- Find movies that are both Comedy and Romance
- Count movies with no genres listed
- Number of movies per genre
- Number of movies released per decade
- Top 10 years with the most movie releases
- Most common genre combinations
- Full-text search on movie titles

RATING ANALYSIS

- Rating distribution (how often each star value is used)
- Overall rating statistics
- Average rating of a specific movie (Toy Story)
- Top 10 most-rated (most popular) movies

RECOMMENDATION QUERIES - QUERY 50

Personalised recommendations for user 1

Top weighted-rated Action/Adventure movies (user 1's favourite genres) that user 1 has NOT rated yet.

aggregate
db.movies
\$match
\$all
\$lookup
\$ne
\$addFields
\$size
\$avg
\$gte
\$sort
\$limit
\$project
\$round

```

db.movies.aggregate([
  { $match: { genreList: { $all: ["Action", "Adventure"] } } },
  {
    $lookup: {
      from: "ratings",
      localField: "movieId",
      foreignField: "movieId",
      as: "r"
    }
  },
  { $match: { "r.userId": { $ne: 1 } } },
  {
    $addFields: {
      ratingCount: { $size: "$r" },
      avgRating: { $avg: "$r.rating" }
    }
  },
  { $match: { ratingCount: { $gte: 50 } } },
  { $sort: { avgRating: -1 } },
  { $limit: 10 },
  { $project: {

```

Run again
33 ms · 10 documents

Chart
Table
JSON



Query page running the collaborative recommendation query

Conclusion

This project helped us understand how MongoDB can be used to store and analyse movie rating data. The MovieLens data was imported into collections, cleaned inside the database and analysed using find queries and aggregation pipelines. We were able to find the most popular and highest rated movies, the average rating of each genre, the most active users, the most used tags and the rating activity over the years. Using \$lookup, \$unwind, \$facet and \$bucket we could answer questions that need data from more than one collection.

We also learnt how recommendations can be made using only database queries, by comparing genres, by finding the other movies liked by the fans of a movie and by using a weighted rating. Creating indexes and checking the query plan with explain() showed how indexes make the queries faster. Finally, connecting MongoDB to a web application showed how the same queries can be used in a real application.

References

11. MongoDB Documentation - MongoDB Manual. <https://www.mongodb.com/docs/manual/>
12. MongoDB Aggregation - MongoDB Documentation.
<https://www.mongodb.com/docs/manual/aggregation/>
13. MongoDB Query and Aggregation Operators.
<https://www.mongodb.com/docs/manual/reference/operator/>
14. MongoDB Indexes. <https://www.mongodb.com/docs/manual/indexes/>
15. MongoDB Shell (mongosh) Documentation. <https://www.mongodb.com/docs/mongodb-shell/>
16. MongoDB Node.js Driver. <https://www.mongodb.com/docs/drivers/node/current/>
17. GroupLens Research - MovieLens Latest Datasets.
<https://grouplens.org/datasets/movielens/latest/>
18. Kaggle - Popular Movies Datasets (9742 Movies).
<https://www.kaggle.com/datasets/whenamancodes/popular-movies-datasets-9000-movies>
19. F. Maxwell Harper and Joseph A. Konstan. The MovieLens Datasets: History and Context. ACM Transactions on Interactive Intelligent Systems, 2015.
20. Express.js Documentation. <https://expressjs.com/>
21. React Documentation. <https://react.dev/>